

Elective Surgery Scheduling

Interval-based CP-SAT model for weekly OR planning

Hector Bonilla

Agenda

- 1 Problem & Motivation
- 2 Assumptions & Scope
- 3 Data & Instances
- 4 The CP-SAT Model
- 5 Why CP-SAT, Not a MIP
- 6 Results
- 7 Production Deployment
- 8 Architecture & Reusability

Main flow: full CP-SAT model including all variables, parameters, and constraints. Comparison MIP stays in the appendix – it's evidence for the CP-SAT choice, not a second deliverable.

The problem

One hospital, one week. An elective waiting list always bigger than the week's OR capacity.

- For each waiting case: which room, which day, what clock time, which surgeon – or defer to the following week.
- **Feasibility** constraint: rooms, surgeons, shared equipment, and downstream recovery beds are all finite and shared.
- **Fairness** constraint: clinical urgency, not FIFO order, should determine priority – a genuinely urgent patient must not sit behind a routine one merely because of arrival order.
- **Scope (Cardoen et al., 2010)**: this is *advance scheduling* – assigning cases to days/rooms ahead of the session – distinct from same-day sequencing or real-time disruption handling.

Why it matters

- **OR time is expensive and perishable.** $\sim$ \$30–40/minute once staffing, equipment and overhead are amortised (Macario, 2010). An idle OR-minute today cannot be banked for tomorrow.
- **The waiting list is the audited metric.** In systems like Portugal's SIGIC, the UK NHS RTT, and Canadian provincial benchmarks, waiting-list breach rates are the number that gets published and that planners are graded on – not just a side metric.
- **Elective and emergency demand share the same rooms** (Van Riet & Demeulemeester, 2015), making capacity allocation a genuine strategic decision, not a by-product of scheduling.
- **Evidence from the literature:** Cardoen, Demeulemeester & Beliën (2010) survey over 200 papers; advance scheduling under priority/deadline constraints is its own well-defined sub-problem, with this project occupying its core scope.

The Portuguese case: the SIGIC system

SIGIC – *Sistema Integrado de Gestão de Inscritos para Cirurgia* – is Portugal's national waiting-list management system, established by **Portaria n.º 45/2008** (Diário da República).

Every elective surgical case is assigned one of **four clinical priority tiers**, each with its own **maximum acceptable wait**:

Priority	Clinical meaning	Max wait (days)
1	Routine elective	270
2	Elevated urgency	60
3	Urgent	15
4	Emergent add-on – run this week	3

These four numbers are *not* illustrative defaults invented for this demo – they are the model's wl_p^{max} parameters, taken directly from a real, legislated, audited policy.

The Portuguese case: what the 2016 CHLN audit found

Marques & Captivo (2015) audited the Centro Hospitalar Lisboa Norte (CHLN) waiting list against SIGIC targets. Key findings:

- **~7,400** patients on the list at audit time.
- **16%** had *already* breached their tier's maximum-wait deadline.
- Average overrun among breached cases: **147 days** hospital-wide.
- Neurosurgery: average overrun of **~261 days** – far worse than the hospital-wide average.

The Portuguese case: what the 2016 CHLN audit found

Marques & Captivo (2015) audited the Centro Hospitalar Lisboa Norte (CHLN) waiting list against SIGIC targets. Key findings:

- **~7,400** patients on the list at audit time.
- **16%** had *already* breached their tier's maximum-wait deadline.
- Average overrun among breached cases: **147 days** hospital-wide.
- Neurosurgery: average overrun of **~261 days** – far worse than the hospital-wide average.

Why this shapes this model's objective

A planner optimising only throughput can report excellent utilization while quietly reproducing this audit's failure mode – scheduling routine cases and deferring overdue ones indefinitely, because a FIFO queue or an unweighted objective can't distinguish them. The penalty term w_c in the objective exists specifically to make breach costs *explicit, weighted, and dominant*.

What the model covers – and what it doesn't

Modelled:

- Case→room→time→surgeon assignment
- Room/service rosters (block schedule)
- Surgeon availability, daily & weekly limits
- One shared imaging unit (C-arm) with capacity
- Recovery/ICU bed pool (multi-day stay)
- Priority-4 emergency add-on hard lock-in
- Pediatric-block carve-out rule

Deliberately excluded:

- Stochastic durations
- Same-day disruptions
- Nurse/anaesthetist rostering
- Sequence-dependent OR cleaning
- Multi-week rolling horizon
- Emergency capacity reserve sizing
- Patient day-of-week preferences

Key assumptions – detail and justification

- 1 **Deterministic durations.** One duration per case (e.g. historical median for that procedure). Real durations are stochastic and biased *long*; the deterministic case is the standard tractable approximation (Denton et al., 2010). Stochastic extension requires two-stage SP – top future-work item.
- 2 **Turnover bucketed by duration** (15/25/40 min): proxy for the real 15–60 min range. Captures the duration-correlated component; not the sequence-dependent one (that's CP Optimizer, backup appendix).
- 3 **Surgeons as binding staffing resource.** Nurses / anaesthetists assumed bundled with the rostered room – valid when the surgeon's calendar is the actual bottleneck.
- 4 **One occurrence per patient per week.** Conservative default (combining same-day procedures is service-specific clinical judgment, not assumed here).
- 5 **Constant bed capacity across the week**, with explicit overflow charge for stays crossing the horizon – no silent approximation.
- 6 **Pediatric block** as a worked example of an ad hoc institutional rule: one eligibility predicate, no new variables.
- 7 **Single week, single hospital, no same-day disruption** – offline advance scheduling only.

Data model

The data layer (`src/model/types.py`) is solver-agnostic – plain Python dataclasses, no solver imports.

- **SurgicalCase**: patient id, service, surgeon, priority p_c , days waited wl_c , operative time t_c^{op} , turnover t_c^{clean} , optional equipment, optional bed pool.
- **Surgeon**: daily limit k_{hd} , weekly limit k_h , per-day availability.
- **OperatingRoom**: per-day capacity k_{dr} , service roster, ambulatory-only flag.
- **PlanningInstance**: collects all the above plus policy parameters (wl_p^{max} , μ_p , α , equipment/bed capacities, pediatric-block rule).

Every solver reads **PlanningInstance** and returns **SolverResult** – the solver adapter pattern that makes backends swappable.

demo_instance() – 20 cases, 5 rooms, 6 surgeons

3 services: ENT, ORTHO, VASC.

Service	Rooms	Capacity (min/day)	Surgeons
ENT	2	360 ($\approx$ 6 h)	2
ORTHO	1	450 ($\approx$ 7.5 h)	1
VASC	2	660 ($\approx$ 11 h)	3

Special features activated (one per constraint family):

- 3 priority-4 cases (C05 ENT, C13 ORTHO, C19 VASC) – hard lock-in to Monday
- Shared C-arm with capacity 1/day (VASC endovascular cases: C14, C15, C17, C19)
- 2 ICU beds/day; C14 & C16 need a 1-day ICU stay
- Pediatric block: Friday ENT rooms restricted to age ≤ 8
- 3 overdue cases (C03, C11, C16) – already past their SIGIC deadline

Sized to be read by eye and to exercise every constraint family in one run.

medium_instance() – 200 cases, 12 rooms, 17 surgeons

5 services; structure loosely modelled on Cardoen et al. (2010) multi-service benchmark family.

Service	Rooms	Cap (min/d)	Surgeons	Equipment
ENT	2	360	3	–
ORTHO	3	480	4	–
VASC	2	660	3	C-ARM
GYN	2	300	3	–
NEURO	3	540	4	C-ARM

- Priority mix: 58% P1 / 26% P2 / 13% P3 / 3% P4 (realistic elective distribution)
- ~40% of surgeons have one day off per week (Monday never)
- 12% of VASC/NEURO cases need an ICU bed (1–2 days)
- Surgeon limits: 300 min/day, 1300 min/week (larger-hospital framing)

Used to demonstrate scaling and make the CP-vs-MIP gap measurable (results section).

Real-data availability

The synthetic instances above are *structurally* calibrated to the literature. Two openly-licensed real hospital datasets exist that map directly onto `PlanningInstance` without any formulation change – only a loader is missing:

- Akbarzadeh & Maenhout (2023a). *Real life data for OR scheduling* – Ghent University Hospital OR log, May 2017. Mendeley Data V2. doi:10.17632/n2v49z2vnp.2
- Akbarzadeh & Maenhout (2023b). *RealLife OR scheduling dataset, 2021-Jan-May* – 20 weekly instances, 8 demand/flexibility configurations. Mendeley Data V1. doi:10.17632/c8d342266x.1

Both are CC BY 4.0 and contain the same master-roster + per-case waiting-list schema used by `PlanningInstance`. A production deployment would start here.

Sets and indices

Symbol	Meaning
$c \in C$	Surgical cases (one entry per patient-procedure pair)
$d \in D$	Planning days, $D = \{1, \dots, 5\}$ (Mon–Fri)
$r \in R$	Operating rooms
$h \in H$	Surgeons
$e \in E$	Shared equipment types (one unit per type in the demo)
$D_c \subseteq D$	Eligible days for case c ; $\{1\}$ if priority-4

Candidates (c, d, r) are pre-filtered by eligibility (C4–C6) *before* any variable is created – this is the model's main variable-count reduction, not a constraint row.

Parameters (I): time, capacity, priority

Symbol	Meaning	Default / source
t_c^{op}	Operative duration (min)	Historical median
t_c^{clean}	Turnover (min)	15/25/40 by duration bucket
$t_c^{tot} = t_c^{op} + t_c^{clean}$	Total room occupation	—
k_{dr}	Room r open minutes on day d	Hospital roster
k_{hd}	Surgeon daily op. limit (min)	240 (demo) / 300 (med)
k_h	Surgeon weekly op. limit (min)	960 (demo) / 1300 (med)
$p_c \in \{1, 2, 3, 4\}$	Clinical priority tier	SIGIC triage
wl_c	Days waited (planning date)	Hospital data
wl_p^{max}	Max acceptable wait for tier p	270/60/15/3 (SIGIC)
$dd_c = wl_p^{max} - wl_c$	Slack to deadline (< 0 = overdue)	Computed

Parameters (II): objective and shared resources

Symbol	Meaning	Default / source
μ_p	Priority multiplier	wl_1^{max}/wl_p^{max} (derived, see below)
w_c	Non-scheduling penalty	$\mu_{p_c} \cdot \text{PenaltyCurve}(dd_c) + 1.2 \max dd$
$\alpha > 1$	Urgency amplifier for overdue cases	2.0 (day-ordering knob)
$u_{ce} \in \{0, 1\}$	1 if case c needs equipment e	Instance data
κ_{ed}	Simultaneous capacity of equip. e on day d	1 (demo), 2 (med)
$\rho(c)$	Recovery bed pool required ("none" if not needed)	Instance data
los_c	Length of stay in that pool (days)	Instance data
β_ρ	Bed count in pool ρ (constant/week)	2 (demo), 6 (med)
π^{ovf}	Penalty/day for bed stay past horizon	50 (configurable)

μ_p : tier allowed only 1/12 of P1's wait is $12\times$ more wait-sensitive: $\mu_2 = 270/60 = 4.5$, $\mu_3 = 270/15 = 18$, $\mu_4 = 270/3 = 90$.

Decision variables – presence and scheduling

For every (c, d, r) surviving eligibility:

$$\text{pr}_{cdr} \in \{0, 1\}, \quad \text{start}_{cdr}, \text{end}_{cdr} \in [0, k_{dr}]$$

For every case that is not priority-4:

$$u_c \in \{0, 1\}, \quad \sum_{d \in D_c, r} \text{pr}_{cdr} + u_c = 1$$

- $\text{pr}_{cdr} = 1$: case c is scheduled on day d in room r .
- $u_c = 1$: case c is *not* scheduled this week and incurs penalty w_c .
- Priority-4 cases have no u_c : they are forced to be scheduled (constraint C2 handles this).
- Only (c, d, r) triples passing C4–C6 eligibility get a variable – the main variable-count reduction.

Decision variables – two interval variables per candidate

$iv_{cdr} = \text{NewOptionalIntervalVar}(\text{start}_{cdr}, t_c^{tot}, \text{end}_{cdr}, \text{pr}_{cdr}) (\text{ROOM})$

$sgiv_{cdr} = \text{NewOptionalIntervalVar}(\text{start}_{cdr}, t_c^{op}, \text{sgend}_{cdr}, \text{pr}_{cdr}) (\text{SURGEON})$

Both share start; both are *present* (constrain their resource) exactly when $\text{pr}_{cdr} = 1$.

Decision variables – two interval variables per candidate

$$iv_{cdr} = \text{NewOptionalIntervalVar}(\text{start}_{cdr}, t_c^{tot}, \text{end}_{cdr}, \text{pr}_{cdr}) (\text{ROOM})$$

$$\text{sgiv}_{cdr} = \text{NewOptionalIntervalVar}(\text{start}_{cdr}, t_c^{op}, \text{sgend}_{cdr}, \text{pr}_{cdr}) (\text{SURGEON})$$

Both share start; both are *present* (constrain their resource) exactly when $\text{pr}_{cdr} = 1$.

Why two intervals, not one?

Surgeon H operates Case X, $t_c^{op} = 70$, $t_c^{clean} = 20$, starting at $t = 0$. The *room* must stay blocked until $t = 90$. The *surgeon* is free at $t = 70$ and can scrub into Room 2 for Case Y while nursing staff finish cleaning Room 1. Using a single t_c^{tot} -sized interval for C8 would wrongly forbid that – the surgeon would appear busy until $t = 90$ in any room. Separating C7 (room, t_c^{tot}) from C8 (surgeon, t_c^{op}) on the same start resolves this without an explicit channeling constraint.

Objective function – complete formula

$$\begin{aligned}
 \min \quad & \underbrace{\sum_{\substack{c: dd_c \geq 0 \\ d \in D_{c,r}}} [dd_c + d] pr_{cdr}}_{\text{Term 1 – on-time, prefer earlier}} + \underbrace{\sum_{\substack{c: dd_c < 0 \\ d \in D_{c,r}}} [dd_c + \alpha d] pr_{cdr}}_{\text{Term 2 – overdue, urgency-weighted}} \\
 & + \underbrace{\sum_{c: p_c \neq 4} w_c u_c}_{\text{Term 3 – non-scheduling penalty}} + \underbrace{\pi^{ovf} \sum_{c: \rho(c) \neq \text{none}} \text{overflow}_c}_{\text{Term 4 – bed-overflow penalty}}
 \end{aligned}$$

- Term 1: rewards scheduling any case, preferring earlier days.
- Term 2: for overdue cases, $\alpha > 1$ makes the planner front-load them.
- Term 3: dominant term – ensures the model schedules rather than drops.
- Term 4: penalises bed stays running past the week, making cost explicit.

Non-scheduling penalty w_c and priority calibration

$$w_c = \mu_{p_c} \cdot \text{PenaltyCurve}(dd_c) + 1.2 \cdot \max_{c' \in C} dd_{c'}$$

PenaltyCurve (`src/model/penalty.py`): flat while the case has slack, escalates sharply on breach, keeps rising with backlog – the shape SIGIC-style systems use to price breach increasingly over time.

Priority multipliers μ_p – *derived* from SIGIC:

$$\mu_p = \frac{wl_1^{max}}{wl_p^{max}} \Rightarrow \mu_2 = \frac{270}{60} = 4.5, \quad \mu_3 = \frac{270}{15} = 18, \quad \mu_4 = \frac{270}{3} = 90$$

A tier allowed only 1/12 of P1's wait is, by that *same* policy, 12× more wait-sensitive.

Displacement margin 1.2: the minimum needed is $1 + \alpha n_{\text{days}} / \max dd_c \approx 1.037$; 1.2 gives a safety buffer across instances with shorter max-*dd*.

Constraints C1–C3: case disposition

C1 – at most one occurrence per patient per week:

$$\sum_{c: \text{patient}(c)=n} \sum_{d \in D_{c,r}} \text{pr}_{cdr} \leq 1 \quad \forall n$$

C2 – priority-4 locked to day 1:

$$\sum_r \text{pr}_{c,1,r} = 1 \quad \forall c : p_c = 4 \quad (\text{all other slots for } c \text{ forced to } 0)$$

C3 – every other case is scheduled exactly once or penalised:

$$\sum_{d \in D_{c,r}} \text{pr}_{cdr} + u_c = 1 \quad \forall c : p_c \neq 4$$

C4–C6 (eligibility): room-service roster, ambulatory-only flag, pediatric-block age rule – all three are resolved during candidate generation; they never appear as a constraint row and add no overhead to the solver.

Constraints C7–C9: room and surgeon

C7 – room, exact non-overlap:

$$\text{AddNoOverlap}(\{\text{iv}_{cdr} : d, r \text{ fixed}\}) \quad \forall d \in D, r \in R$$

C8 – surgeon, exact non-overlap on *surgeon's* interval + daily cap:

$$\text{AddNoOverlap}(\{\text{sgiv}_{cdr} : \text{surgeon}(c) = h, d \text{ fixed}\}) \quad \forall h, d$$

$$\sum_{c: \text{surgeon}(c)=h} \sum_r t_c^{op} \text{pr}_{cdr} \leq k_{hd} \quad \forall h \in H, d \in D$$

C9 – surgeon weekly limit:

$$\sum_{c: \text{surgeon}(c)=h} \sum_{d,r} t_c^{op} \text{pr}_{cdr} \leq k_h \quad \forall h \in H$$

NoOverlap bounds concurrency; C8/C9 cap total hours – both are needed because they constrain different things.

Constraints C10–C11: equipment and beds

C10 – shared equipment, exact time-overlap check:

$$\text{AddCumulative}(\{iv_{cdr} : u_{ce} = 1, d \text{ fixed}\}, \mathbf{1}, \kappa_{ed}) \quad \forall e \in E, d \in D$$

C11 – recovery/ICU beds, with horizon-overflow charge:

$$\text{dayof}_c = d_{\text{idx}} \text{ (via OnlyEnforceIf when } pr_{cdr} = 1)$$

$$\text{bed}_c = \text{NewOptionalIntervalVar}(\text{dayof}_c, \text{los}_c, \text{dayof}_c + \text{los}_c, 1 - u_c)$$

$$\text{AddCumulative}(\{\text{bed}_c : \rho(c) = \rho\}, \mathbf{1}, \beta_\rho) \quad \forall \rho$$

$$\text{overflow}_c = \max(0, \text{dayof}_c + \text{los}_c - n_{\text{days}})$$

C11 needs a real “day of surgery” value – the concrete reason this model is interval-based. A day-bucket MIP has no such variable (see backup appendix).

CP-SAT search strategy

- **Parallel portfolio** – several complete-search workers with different heuristics, plus Large Neighbourhood Search workers improving an incumbent, sharing learned clauses via a common SAT core (Perron & Furnon, Google OR-Tools docs).
- **No custom branching strategy** – OR-Tools' guidance is that the default portfolio outperforms hand-tuned single strategies unless you have structural insight not already exposed through `NoOverlap/Cumulative`. This model exposes that structure fully.
- **Workers** capped at `os.cpu_count()` – over-subscribing past physical cores adds contention, not search diversity.
- **Gap target** = 1% by default; solver stops when the best bound is within 1% of the current incumbent.
- **“Optimal”** means **within the configured gap**, not a literal 0% gap – always printed and never assumed zero.

The core argument

Key distinction

“Total minutes of equipment use today $\leq$ capacity” certifies that a set of durations *fits*. It does *not* certify they can be placed without *colliding*. For one room the two coincide (non-overlapping durations can always be packed sequentially). They **stop coinciding** when a resource is shared across multiple rooms.

MIP (day-bucket): count C-arm cases per day ≤ 1 . Legal placements forbidden: 2 non-overlapping C-arm cases on the same day fail this check even if they never overlap.

CP-SAT: AddCumulative checks literal time overlap. Two sequential C-arm cases on the same day – never concurrent – pass the check. Larger feasible region, better possible schedules.

Advantages and disadvantages

CP-SAT / advantages:

- NoOverlap/Cumulative with poly-time propagation (Vilím 2004; Schutt et al. 2009)
- Exact intra-day clock times modelled natively
- Bed constraint C11 expressible at all
- Parallel portfolio search (LNS + complete search)
- No big-M constants, no LP-relaxation weakening

CP-SAT / disadvantages:

- Own gap can be looser (larger search space)
- Less interpretable as a “classical” LP

MILP / advantages:

- Strong LP relaxation bounding via cutting-plane theory
- Mature commercial solvers (Gurobi, CPLEX)
- Proven MIP gap closed very fast on small-to-medium instances
- Own gap closes more tightly on a smaller feasible region

MILP / disadvantages:

- Quadratic variable growth for disjunctions
- Big-M weakens LP relaxation
- Cannot express C11 (multi-day bed interval)
- Day-count equipment cap *logically incorrect* for shared resources

Demo instance results – summary table

```
python main.py --instance demo --benchmark --gap 0.0001
```

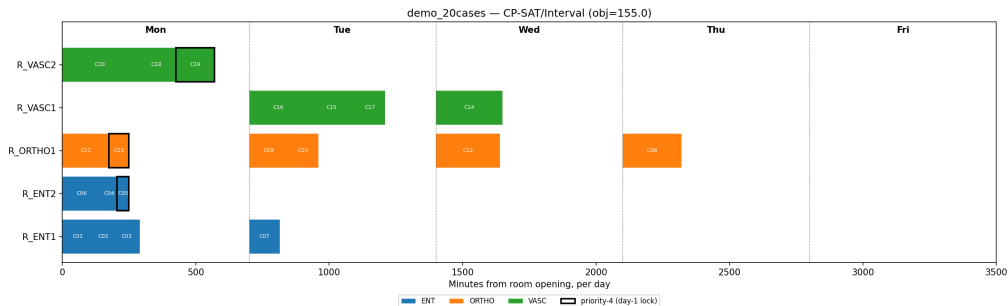
Solver	Status	Objective	Gap	Scheduled
CP-SAT (primary)	Optimal	155.0	0.00%	20/20
OR-Tools/CBC (MIP)	Optimal	157.0	0.00%	20/20
Gurobi (MIP)	Optimal	157.0	0.00%	20/20
CP Optimizer	Optimal	155.0	0.00%	20/20

C-arm cases (4 in total: C14, C15, C17, C19):

- **CP-SAT:** Mon=[C19], Tue=[C15, C17], Wed=[C14] – two C-arm cases placed on the *same day* sequentially.
- **MIP:** Mon=[C19], Tue=[C15], Wed=[C17], Thu=[C14] – one C-arm per day, forced by the day-count cap.

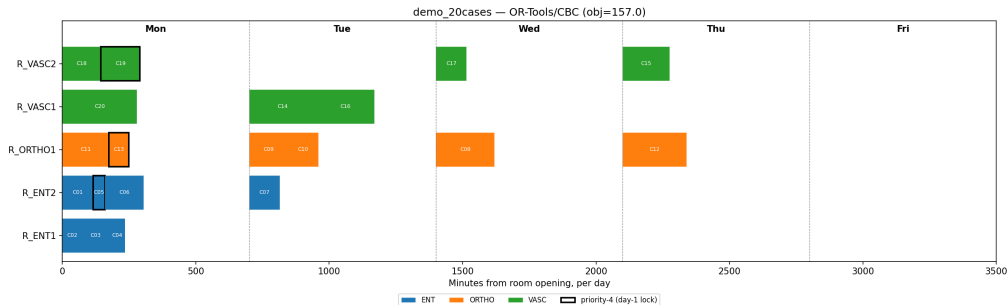
The objective difference (155 vs. 157) comes directly from this placement: CP-SAT schedules the same 20 cases with more efficient equipment use.

Demo – CP-SAT Gantt schedule



Reading the chart: each row is one room; each bar is one case, coloured by service. Outlined bars (bold edge) are priority-4 cases locked to Monday. Note **Tuesday**: Room R_VASC1 and R_VASC2 each carry one C-arm case sequentially – the scheduler has correctly identified they never overlap in time, even though both land on the same day.

Demo – Comparison MIP Gantt schedule



Reading the chart: same cases, same services. The MIP has no exact clock times (day-bucket formulation) – cases are laid out back-to-back for display only, the solver makes no claim about their order. Compare the C-arm distribution: each of Monday, Tuesday, Wednesday, Thursday carries exactly one C-arm case. The day-count constraint $\sum x_{cdr} \leq 1$ is satisfied, but it produces an unnecessarily spread schedule – a smaller feasible region, not a worse search.

Medium instance results (200 cases, 2-minute budget)

Step 1 – true MIP optimum via Gurobi (near-zero gap): **74,074.0**, 130/200 scheduled, in < 1 second.

Step 2 – realistic 2-minute / 1% gap budget:

Solver	Status	Objective	Own gap	vs. true opt.	Sched.
Gurobi (reference)	Optimal	74,074.0	0.00%	—	130/200
OR-Tools/CBC	Feasible	74,116.0	0.56%	+0.06%	130/200
CP-SAT	Feasible	69,956.0	6.62%	−5.56%	131/200

CBC reaches the MIP's own proven optimum in 2 minutes. CP-SAT **beats that optimum by 5.56%** while scheduling one more case – it is searching a strictly larger, correct feasible region, not a worse search. CP-SAT's own gap (6.62%) is *relative to its own* feasible region's bound – a larger space is harder to close, the same reason it's harder to prove there's no number above 5 in $\{1, \dots, 100\}$ than in $\{1, \dots, 5\}$.

Medium instance – CP-SAT Gantt schedule (200 cases)



What would change for a real deployment

- 1 **Data loader:** map the hospital's EHR/PAS export to `PlanningInstance`. Schema aligns with Akbarzadeh & Maenhout (2023) CC BY 4.0 datasets – the loader is the only missing piece.
- 2 **Parameter calibration:** replace synthetic defaults with hospital-specific values:
 - w_p^{max} : hospital's own published wait targets
 - k_{hd}/k_h : surgeon job-plan session lengths
 - μ_p : structured elicitation with service chiefs (scenario-anchored, not multiplier-level)
 - β_ρ, κ_{ed} : actual bed census & equipment inventory
- 3 **Weekly cadence:** run with a 30-minute to 2-hour budget; 2-minute results already shown to be 5.56% below the MIP's proven optimum, demonstrating strong performance even at short budgets.
- 4 **Acceptance testing:** extend `tests/test_model.py` with a real-data backtest; any reimplementations must pass the same hard-constraint checks on the demo instance.
- 5 **Fairness monitoring:** track per-service overdue share alongside the aggregate objective (the μ_p equity caveat from the calibration section).

How the project is built

- **CP-SAT** (`cp_sat_interval_solver.py`) – the model, default backend, only one needed to run.
- **Comparison MIP** (`milp_baseline_solver.py`) – CBC / SCIP / Gurobi / CPLEX via `--solver milp-cbc` etc.; kept purely as empirical evidence for the CP choice.
- **CP Optimizer** (`cp_optimizer_solver.py`) – optional, license-gated; adds sequence-dependent turnover; auto-falls-back to CP-SAT.
- `src/model/types.py` – solver-agnostic data dictionary; every mathematical symbol maps to one field here.
- `src/model/penalty.py` – one implementation of w_c , shared by every backend.
- `tests/test_model.py` – acceptance contract: hard constraints verified on every solver's output on every run.

A reusable library of models (Q&A answer 2)

- 1 **Core data types** – typed dataclasses, no solver imports; every model in the library builds on top of them.
- 2 **Constraint patterns** – capacity sums, NoOverlap-based non-overlap, tiered-priority tardiness, eligibility pre-filters. Nurse rostering and bed allocation need the same *shapes*, not the same model.
- 3 **Problem templates** – compose those patterns into a specific formulation. This project is one template.
- 4 **Solver-adapter layer** – one file per backend family (MILP, CP, local search); a new problem picks a backend without rewriting constraint logic.

The CP-vs-MIP empirical comparison this project runs is itself the template for layer 4: *argue* the backend choice from the problem's structure, *verify* it on a small instance, then commit – rather than defaulting to whichever backend the team knows best.

Passing the torch (Q&A answer 1)

- 1 **FORMULATION.md + FORMULATION_CP.md**: problem framing, evidence, assumptions, all sets/params/variables/constraints with full math. One source of truth.
- 2 **src/model/types.py**: the dataclasses *are* the data dictionary – every mathematical symbol maps to exactly one field, no drift possible.
- 3 **Solver code with C-numbers**: every constraint in `cp_sat_interval_solver.py` is labeled C1...C11, matching FORMULATION_CP.md – read side by side, no ambiguity.
- 4 **tests/test_model.py**: acceptance bar – any reimplementations must pass the same hard-constraint checks on the demo instance; write a new test alongside any new constraint.
- 5 **A short glossary**: most confusion on a project like this is vocabulary (“room roster”, “ambulatory”, what P4 operationally means), not the math.

Where I'd take this next

Extension	Approach
Stochastic durations	Two-stage SP; 1st stage places, 2nd absorbs duration draws
Same-day rescheduling	LNS seeded from current plan
Explicit emergency reserve	Per-day capacity block outside the elective model
Nurse/anaesthetist rostering	Extend H , reuse surgeon NoOverlap/cap pattern
Multi-week rolling horizon	Carry forward unscheduled at bumped priority
Day-varying bed capacity	Per-day-segmented Cumulative
Per-specialty fairness	Secondary objective bounding per-service overdue share
Real-data pilot	Akbarzadeh & Maenhout CC BY 4.0 datasets

Questions

Thank you.

▶ Back to agenda

▶ Full MIP appendix

MIP – formulation overview

The comparison MIP (FORMULATION.md, Appendix A) is implemented in `milp_baseline_solver.py` and runs via `--solver milp-cbc` (bundled), `milp-gurobi`, or `milp-cplex` (commercial license required).

Same as CP-SAT: sets, eligibility pre-filter (C4–C6), priority tiers, w_c from the same `penalty.py`.

Key differences:

- No intra-day clock times – decisions are at *day+room* granularity only.
- C7 and C10 are linear capacity sums, not exact non-overlap.
- C11 (recovery beds) is *not expressible* – a day-bucket variable has no “day of surgery” value for a multi-day stay.
- No Term 4 in the objective (overflow penalty) – C11 doesn't exist.

MIP – variables and objective

$$x_{cdr} \in \{0, 1\} \forall c, d \in D_c, r \quad z_c \in [0, 1] \forall c : p_c \neq 4$$

z_c is relaxed to continuous; C3 forces it to $\{0, 1\}$ at the optimum (nothing else benefits from declaring it binary).

$$\min \sum_{c: dd_c \geq 0} \sum_{d, r} [dd_c + d] x_{cdr} + \sum_{c: dd_c < 0} \sum_{d, r} [dd_c + \alpha d] x_{cdr} + \sum_{c: p_c \neq 4} w_c z_c$$

Same three-term structure as CP-SAT Terms 1–3; the same `penalty.py` function computes w_c . No Term 4 (no C11).

◀ Agenda

MIP – constraints and what's missing

C7 (room, aggregate sum):

$$\sum_{c \in C} t_c^{tot} x_{cdr} \leq k_{dr} \quad \forall d, r$$

For a single room: equivalent to exact non-overlap. For a *shared resource across rooms*: incorrectly counts all equipment-uses as if they overlap.

C10 (equipment, daily headcount):

$$\sum_{c: u_{ce}=1} \sum_r x_{cdr} \leq \kappa_{ed} \quad \forall e, d$$

Counts *how many* equipment-*e* cases land on a day, not whether they overlap in clock time – the main source of the 5.56% objective gap vs. CP-SAT at scale.

No C11: “bed_{*c*} starts on whichever day *c* is scheduled” cannot be written – no variable in a day-bucket model holds “day of surgery” as a *value*.

CP Optimizer (optional, appendix)

Kept for one reason: **sequence-dependent room turnover** – a transition matrix between adjacent cases in a room via `sequence_var + no_overlap(transition_matrix)`.

	Same service	Different service
CP-SAT (duration-bucketed)	case's own t_c^{clean}	same
CP Optimizer	15 min	35 min

Demo instance: identical to CP-SAT (155.0, 20/20, Optimal) – confirmed that same-service turnover gaps equal exactly 15 min.

At 200 cases, own gap closes slower than CP-SAT (search-tuning difference – no warm start applied; not a modelling weakness).

◀ Agenda

References

- 1 Cardoen, B., Demeulemeester, E., & Beliën, J. (2010). Operating room planning and scheduling: A literature review. *EJOR*, 201(3), 921–932. doi:10.1016/j.ejor.2009.04.011
- 2 Marques, I., & Captivo, M.E. (2015). *Planeamento de cirurgias eletivas no Centro Hospitalar Lisboa Norte*. MSc thesis, Universidade de Lisboa.
- 3 Denton, B.T., Miller, A.J., Balasubramanian, H.J., & Huschka, T.R. (2010). Optimal allocation of surgery blocks to operating rooms under uncertainty. *Oper. Res.*, 58(4), 802–816. doi:10.1287/opre.1090.0791
- 4 SIGIC – Portaria n.º 45/2008, Diário da República, Portugal.
- 5 Van Riet, C., & Demeulemeester, E. (2015). Trade-offs in operating room planning for electives and emergencies. *OR Spectrum*, 37(1), 59–87. doi:10.1007/s00291-013-0345-2
- 6 Macario, A. (2010). What does one minute of operating room time cost? *J. Clin. Anesth.*, 22(4), 233–236. doi:10.1016/j.jclinane.2010.02.003
- 7 Akbarzadeh, B., & Maenhout, B. (2023a). Real life data for OR scheduling problem. Mendeley Data V2. doi:10.17632/n2v49z2vnp.2
- 8 Akbarzadeh, B., & Maenhout, B. (2023b). RealLife OR scheduling dataset, 2021-Jan-May. Mendeley Data V1. doi:10.17632/c8d342266x.1
- 9 Perron, L., & Furnon, V. *CP-SAT: a CP Solver*. Google OR-Tools. developers.google.com/optimization/cp
- 10 Baptiste, P., Le Pape, C., & Nuijten, W. (2001). *Constraint-Based Scheduling*. Kluwer. doi:10.1007/978-1-4615-1479-4
- 11 Vilím, P. (2004). $O(n \log n)$ filtering for unary resource constraints. *CPAIOR 2004*. doi:10.1007/978-3-540-24664-0_15
- 12 Schutt, A., Feydy, T., Stuckey, P.J., & Wallace, M.G. (2009). Why cumulative decomposition is not as bad as it sounds. *CP 2009*. doi:10.1007/978-3-642-04244-7_45
- 13 Laborie, P. (2009). IBM ILOG CP Optimizer for detailed scheduling illustrated on three problems. *CPAIOR 2009*. doi:10.1007/978-3-642-01929-6_5